S. Nokeshwaran
Reg No: 192511007

DBMS Mock Interview — Questions & Answers

Database Management Systems: Query Processing, Transactions, Concurrency & Recovery

Q1. Explain the steps of query processing in DBMS. How does a query optimizer decide the best execution plan?

Query processing is the sequence of steps a DBMS takes to translate a high-level SQL query into an efficient, executable low-level plan and then run it.

Steps of Query Processing

- **Parsing and Translation:** The SQL query is checked for syntax and semantic correctness (table/column existence, type checks) and converted into an internal representation such as a relational algebra expression or parse tree.
- **Query Optimization:** The optimizer generates multiple equivalent execution plans (different join orders, access paths, algorithms) and estimates their cost using statistics (table size, index availability, selectivity). The lowest-cost plan is chosen.
- **Query Code Generation / Plan Selection:** The chosen logical plan is converted into a physical execution plan with concrete algorithms (e.g., nested-loop join vs. hash join) and access methods (index scan vs. full scan).
- **Execution (Evaluation Engine):** The runtime engine executes the physical plan against the stored data and returns the result set to the user/application.

How the Optimizer Decides the Best Plan

- It enumerates alternative plans (join orders, algorithms, index usage) using equivalence rules of relational algebra.
- It estimates the **cost** of each plan (in terms of disk I/O, CPU, and memory) using catalog statistics such as table cardinality, index selectivity, and data distribution histograms.
- It uses **dynamic programming** or heuristics (e.g., pushing selections and projections down early) to prune the search space instead of evaluating every possible plan.
- The plan with the minimum estimated cost is selected and handed to the execution engine.

Q2. What is the difference between heuristic optimization and cost-based optimization? When would you use each?

Aspect	Heuristic (Rule-Based) Optimization	Cost-Based Optimization
Basis	Fixed transformation rules / heuristics	Estimated execution cost using statistics
Statistics needed	Not required	Requires table/index statistics
Examples of rules used	Push selection & projection early, avoid Cartesian products, perform most restrictive joins first	Compare alternative join algorithms, join orders, access path
Speed of optimization	Fast, low overhead	Slower, more computation during planning
Plan quality	Generally good but not guaranteed optimal	Usually closer to optimal since it factors in actual data characteris
Best suited for	Simple queries, systems with no statistics, or as a first-pass simplification step	Complex queries with multiple joins, large tables, modern product

When to use each: Heuristic optimization is preferred in lightweight or embedded databases where collecting and maintaining statistics is impractical, or as an initial simplification before deeper analysis.

Cost-based optimization is preferred in enterprise/production systems handling complex multi-join queries over large, frequently changing datasets, where accurate statistics allow the optimizer to genuinely pick the cheapest plan. In practice, most modern optimizers use a hybrid: heuristics to prune obviously bad plans, then cost-based comparison among the survivors.

Q3. How would you implement database connectivity in a real project? Explain the difference between JDBC and ODBC.

In a real project, database connectivity is implemented through a standard API/driver layer that sits between the application code and the database server, so the application does not need to know the database's internal protocol.

Typical Implementation Approach

- Add the appropriate driver/connector library for the database (e.g., a JDBC driver jar for MySQL/PostgreSQL in Java, or a connector package in Python/.NET).
- Configure connection parameters (host, port, database name, credentials) — ideally through a config file or environment variables, not hard-coded.
- Use a **connection pool** (e.g., HikariCP, Apache DBCP) to avoid the overhead of opening/closing a new connection per request.
- Execute queries using prepared statements to prevent SQL injection and to allow query plan reuse.
- Handle transactions explicitly (commit/rollback) for operations that must be atomic, and always close/release connections back to the pool (try-with-resources or equivalent).

JDBC vs ODBC

Aspect	JDBC (Java Database Connectivity)	ODBC (Open Database Connectivity)
Language	Java-specific API	Language-independent (C-based API), usable from C/C++, and via
Platform	Platform-independent (runs on JVM)	Platform-dependent; needs native drivers per OS
Driver type	Pure Java drivers (Type 4 most common) communicate directly with DB server	Native binary drivers registered with the OS driver manager
Performance	Slightly better in Java apps due to no native bridging	Can be faster for native (non-Java) applications since there's no JV
Ease of deployment	Just add the JAR to classpath	Requires driver installation/configuration at OS level (DSN setup)

In short: JDBC is Java's own API for DB access, while ODBC is a universal, OS-level standard usable from many languages. In a real project, you choose JDBC for Java/Spring applications, and ODBC (or a language-native connector) for C/C++/.NET or legacy desktop applications.

Q4. What are ACID properties? Give a real-world example where each property is critical.

ACID properties guarantee that database transactions are processed reliably, even in the presence of errors, power failures, or concurrent access.

- **Atomicity** — A transaction is treated as a single indivisible unit; either all its operations succeed, or none do. *Example:* In a bank fund transfer, debiting one account and crediting another must both happen, or neither — otherwise money could vanish if the system crashes mid-transfer.
- **Consistency** — A transaction moves the database from one valid state to another, preserving all defined rules and constraints. *Example:* An airline seat-booking system must never allow the number of booked seats to exceed the aircraft's capacity; every transaction must respect this constraint.

- **Isolation** — Concurrent transactions execute as if they were run one after another, without interfering with each other. *Example:* Two customers booking the last available movie ticket simultaneously must not both succeed; isolation ensures only one booking goes through.
- **Durability** — Once a transaction is committed, its effects persist even if the system crashes immediately afterward. *Example:* After an ATM withdrawal is confirmed and cash is dispensed, the account balance update must survive a subsequent power failure at the bank's data center.

Q5. Explain concurrency control problems: dirty read, lost update, and unrepeatable read. How does 2PL solve them?

Concurrency Control Problems

- **Dirty Read:** A transaction reads data written by another transaction that has not yet committed. If the writing transaction later rolls back, the reader has used invalid/nonexistent data.
- **Lost Update:** Two transactions read the same data and both update it based on the value they read; one update overwrites the other, so the effect of the first update is silently lost.
- **Unrepeatable Read:** A transaction reads the same row twice and gets different values because another committed transaction modified that row in between the two reads.

How Two-Phase Locking (2PL) Solves Them

2PL requires every transaction to acquire all the locks it needs (Growing Phase) before releasing any lock (Shrinking Phase) — once a transaction releases a lock, it cannot acquire new ones.

- **Prevents dirty reads:** A transaction must hold a shared (read) lock to read data, and a writer must hold an exclusive (write) lock; the writer's exclusive lock blocks others from reading uncommitted changes until it releases the lock (in Strict 2PL, this is held until commit/abort).
- **Prevents lost updates:** An exclusive lock on a data item is granted to only one transaction at a time, so a second transaction attempting to update the same item must wait until the first releases its lock, preventing simultaneous blind overwrites.
- **Prevents unrepeatable reads:** Because locks are held (at least until the shrinking phase begins, and until commit in Strict 2PL), no other transaction can modify a value between two reads by the same transaction.

2PL guarantees that the resulting schedules are **conflict-serializable**, which is what eliminates these anomalies.

Q6. What is deadlock in DBMS? Describe the deadlock detection algorithm and at least two prevention strategies.

A deadlock occurs when two or more transactions are each waiting for a lock held by another transaction in the set, forming a cycle of dependencies so that none of them can ever proceed.

Deadlock Detection Algorithm

- The DBMS maintains a **Wait-For Graph (WFG)**, where each node is a transaction and a directed edge Ti $\rightarrow$ Tj means Ti is waiting for a lock held by Tj.
- Periodically (or on each new wait), the system checks the WFG for a **cycle**.
- If a cycle is found, a deadlock exists among the transactions on that cycle.
- The DBMS resolves it by selecting a **victim transaction** (often the youngest, or the one with least work done/fewest locks held) and rolling it back, releasing its locks so the remaining transactions can proceed.

Deadlock Prevention Strategies

- **Wait-Die scheme:** If an older transaction requests a lock held by a younger one, it is allowed to wait; if a younger transaction requests a lock held by an older one, it is aborted ("dies") and restarted later. This ensures no cycle can form since only older transactions wait.
- **Wound-Wait scheme:** If an older transaction requests a lock held by a younger one, the younger transaction is aborted ("wounded"); if a younger transaction requests a lock held by an older one, it waits. This is the converse priority rule and also guarantees no cycles.
- **Lock ordering:** Impose a global ordering on all lockable resources and require every transaction to request locks in that fixed order, which makes circular waiting impossible.
- **Timeout-based prevention:** A transaction that waits longer than a threshold is assumed to be deadlocked and is aborted, avoiding the need to build a wait-for graph at all (simpler but less precise).

Q7. Compare Immediate Update and Deferred Update recovery. In what scenarios is each preferred?

Aspect	Immediate Update	Deferred Update
When DB is modified	Changes are written to the database as soon as they occur, even before commit	Changes are held in a buffer/log and applied to the actual database only after commit
Undo needed?	Yes — uncommitted changes may already be on disk, so UNDO is required for aborted transactions	No UNDO needed, since nothing uncommitted is written to the DB before commit
Redo needed?	Yes, to reapply committed changes that were not yet flushed to disk	Yes, to reapply committed changes that hadn't been written to the DB
Logging requirement	Must log both old (before) and new (after) values	Only needs to log new (after) values, since the DB is never touched before commit
Overhead	Lower memory usage but recovery is more complex (needs UNDO + REDO)	High memory usage to hold changes until commit, but simpler recovery (needs only REDO)

Preference: Deferred Update is preferred in systems where simplicity of recovery matters and enough buffer memory is available to hold pending changes (e.g., short transactions, embedded systems). Immediate Update is preferred in systems handling long-running or high-volume transactions where it is impractical to keep all changes in memory until commit, at the cost of needing a more complete logging and UNDO/REDO recovery mechanism (common in large production RDBMS).

Q8. Explain Shadow Paging. What are its advantages and limitations compared to log-based recovery?

Shadow paging is a recovery technique that avoids logging by maintaining two page tables: the **current page table**, which points to the pages being actively modified by the running transaction, and the **shadow page table**, which points to the original, unmodified pages and represents the last consistent state of the database.

- When a transaction modifies a page, a new copy of that page is written to a fresh disk location; the current page table is updated to point to this new copy, while the shadow page table still points to the old (unmodified) copy.
- If the transaction commits, the shadow page table is simply replaced with the current page table (an atomic pointer switch), making all changes permanent.
- If the transaction aborts or the system crashes before commit, the current page table is simply discarded and the shadow page table (unchanged) is used — the database automatically reverts to its pre-transaction state, with no UNDO needed.

Advantages

- No need for UNDO/REDO logging, which simplifies the recovery algorithm.
- Recovery after a crash is very fast since it only requires discarding the current page table.
- No overhead of writing and flushing log records during normal operation.

Limitations (vs. Log-Based Recovery)

- **Data fragmentation:** Since modified pages are relocated rather than updated in place, related data can become scattered across disk over time.
- **Garbage collection overhead:** Old shadow pages that are no longer needed must be periodically reclaimed, adding extra overhead.
- **Poor support for concurrent transactions:** It is difficult to support multiple concurrent transactions cleanly since each needs a consistent view of the page table, unlike log-based methods which integrate naturally with fine-grained locking.
- **No support for partial rollback:** It is harder to implement savepoints or partial rollbacks within a transaction, which log-based methods handle easily via log record traversal.
- For these reasons, most modern production databases (Oracle, MySQL, PostgreSQL, SQL Server) use log-based (write-ahead logging) recovery rather than shadow paging.

Q9. What is a serializability schedule? How do you test for conflict serializability using a precedence graph?

A **serializable schedule** is a schedule of concurrently executing transactions whose final effect on the database is equivalent to the effect of running those same transactions one at a time, in some serial order. Serializability is the correctness criterion used to allow concurrency while still guaranteeing correctness equivalent to serial execution.

Conflict serializability is a practical, testable form of serializability based on conflicting operations — two operations conflict if they belong to different transactions, access the same data item, and at least one of them is a write.

Testing Conflict Serializability via Precedence (Serialization) Graph

- Create one node in the graph for each transaction in the schedule.
- For every pair of conflicting operations where an operation of Ti precedes a conflicting operation of Tj in the schedule, draw a directed edge Ti $\rightarrow$ Tj.
- After considering all conflicting pairs, examine the resulting graph.
- If the precedence graph is **acyclic**, the schedule is conflict-serializable, and a valid serial order can be obtained via a topological sort of the graph.
- If the graph contains a **cycle**, the schedule is **not** conflict-serializable.

Q10. Describe the Two-Phase Locking protocol variations: Basic 2PL, Strict 2PL, and Rigorous 2PL. Which is most commonly used and why?

- **Basic 2PL:** The transaction is divided into a Growing Phase (locks are only acquired, never released) and a Shrinking Phase (locks are only released, never acquired). Locks can be released as soon as they are no longer needed, even before commit. It guarantees conflict-serializability, but can lead to **cascading rollbacks** — if a transaction releases a lock early and then aborts, other transactions that already read the modified data must also be rolled back.
- **Strict 2PL:** Same growing/shrinking structure as Basic 2PL, but all **exclusive (write) locks** are held until the transaction commits or aborts. This prevents other transactions from reading uncommitted data, thereby avoiding cascading rollbacks, and ensures recoverability.
- **Rigorous 2PL:** The strictest variant — **all locks (both shared and exclusive)** are held until the transaction commits or aborts, not released even between operations. This gives the strongest guarantee: transactions appear to execute in the exact order in which they commit (strict serializability), and it fully avoids cascading rollbacks.

Most Commonly Used

Strict 2PL is the most commonly used variant in real-world database systems (e.g., it underlies locking in Oracle, SQL Server, and MySQL/InnoDB's default isolation behavior). It strikes the best balance: it prevents cascading rollbacks and guarantees recoverability like Rigorous 2PL, but is slightly less restrictive because shared (read) locks may still be released before commit, allowing marginally more concurrency than Rigorous 2PL while remaining simple to implement and reason about.

— End of Document —